BÁO CÁO THỰC HÀNH

Thực hành An toàn Mạng máy tính

Lab 01: Classical Cryptography

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 23/03/2026

Nhóm 11 – NT101.Q22.2

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24520435	Lê Văn Anh Hải	Task 2.1, 2.2	30%
24520613	Nguyễn Thái Hùng	Task 2.3, 2.4	30%
24520782	Lâm Mai Hồ Nhựt Khánh	Task 2.5, 2.6, 2.7	40%

Task 2.1 Bạn hãy viết một chương trình bằng ngôn ngữ lập trình tự chọn để thực hiện mã hóa và giải mã bằng Caesar Cipher.

a. **Cho phép người dùng nhập một khóa và một chuỗi văn bản để thực hiện mã hóa hoặc giải mã.**
b. **Thực hiện brute-force để tìm bản rõ khi chỉ biết ciphertext và hiển thị kết quả giải mã duy nhất và khóa tương ứng. Sau đó, hãy sử dụng chương trình của bạn để giải mã đoạn văn bản mã hóa sau: (https://pastecode.io/s/y3vj3cqe).**

Thiết kế chương trình

Chương trình sẽ có 3 lựa chọn chính cho người dùng:

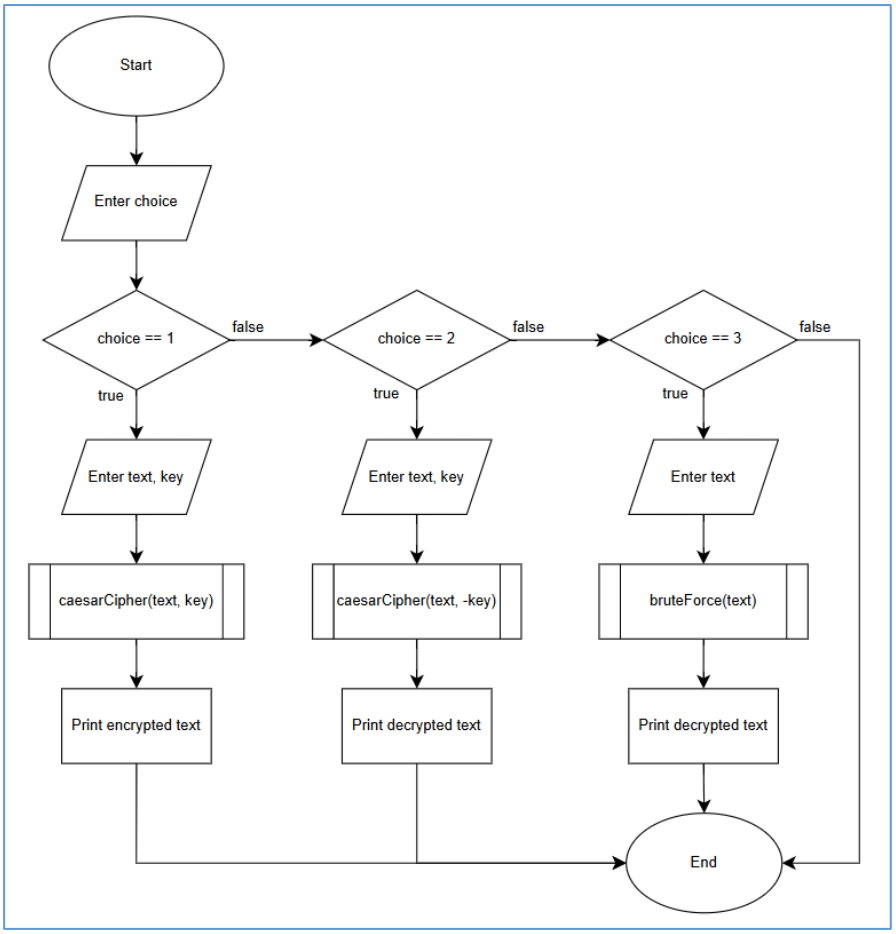
- Mã hóa (Encryption): Người dùng sẽ nhập **plaintext** và **khóa k**, chương trình sẽ thực hiện mã hóa theo công thức:

$$C = (P + k) \; mod \; 26$$

- Giải mã (Decryption): Người dùng sẽ nhập **ciphertext** và **khóa k**, chương trình sẽ thực hiện giải mã theo công thức:

$$P = (C - k) \; mod \; 26$$

- Brute-force: Người dùng sẽ nhập **ciphertext**, chương trình sẽ thử và in ra tất cả các trường hợp **khóa k**, sau đó in ra plaintext hợp lệ.



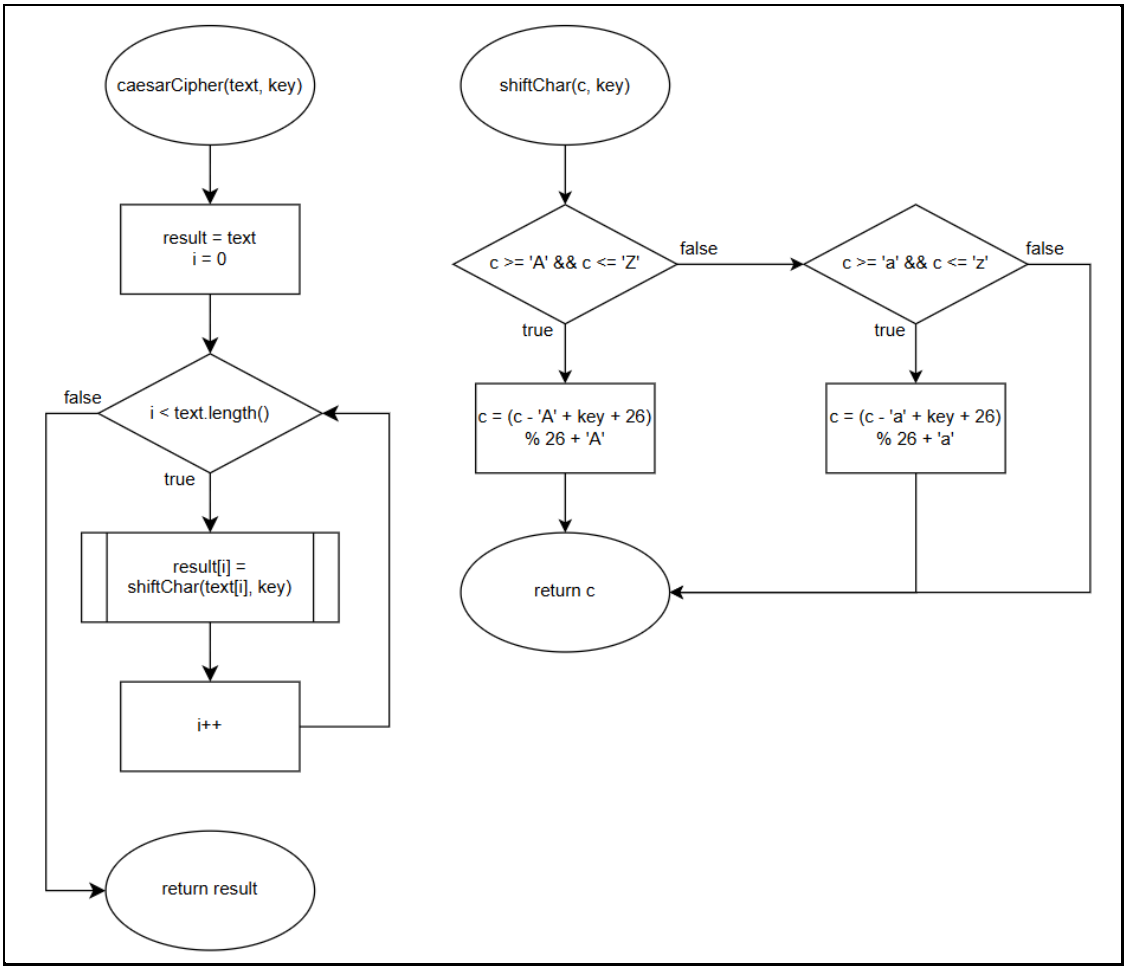
Hình 1. *Flowchart của chương trình Mật mã Caesar.*

Hàm main:

```
int main()
{
    int choice;
    string text;
    int key;

    cout << "===== Caesar Cipher =====" << endl;
    cout << "1. Encrypt" << endl;
    cout << "2. Decrypt" << endl;
    cout << "3. Brute Force" << endl;
    cout << "Choose: ";
    cin >> choice;

    cin.ignore();

    switch(choice)
    {
        case 1:
```

```
        {
            cout << "Enter plaintext: ";
            getline(cin, text);

            cout << "Enter key: ";
            cin >> key;

            cout << "Ciphertext: " << caesarCipher(text, key) << endl;
            break;
        }

        case 2:
        {
            cout << "Enter ciphertext: ";
            getline(cin, text);

            cout << "Enter key: ";
            cin >> key;

            cout << "Plaintext: " << caesarCipher(text, -key) << endl;
            break;
        }

        case 3:
        {
            cout << "Enter ciphertext: ";
            getline(cin, text);

            bruteForce(text);
            break;
        }

        default:
        {
            cout << "Invalid choice!" << endl;
        }
    }

    return 0;
}
```

Hàm **caesarCipher(text, key)** và **shiftChar(c, key)**:



Hình 2. *Flowchart của hàm* ***caesarCipher(text, key)*** *và* ***shiftChar(c, key)****.*

- Hàm **caesarCipher(text, key):**
  - Nhận vào 2 tham số là đoạn text cần xử lý và khóa k để thực hiện dịch chuyển.
  - Duyệt từng ký tự của đoạn text. Gọi hàm shiftChar(c, key) để dịch chuyển từng ký tự theo khóa k.
  - Hàm này sẽ thực hiện mã hóa nếu khóa k được truyền vào là số dương (dịch chuyển ký tự sang phải) và giải mã nếu khóa khóa k được truyền vào là số âm (dịch chuyển ký tự sang trái).

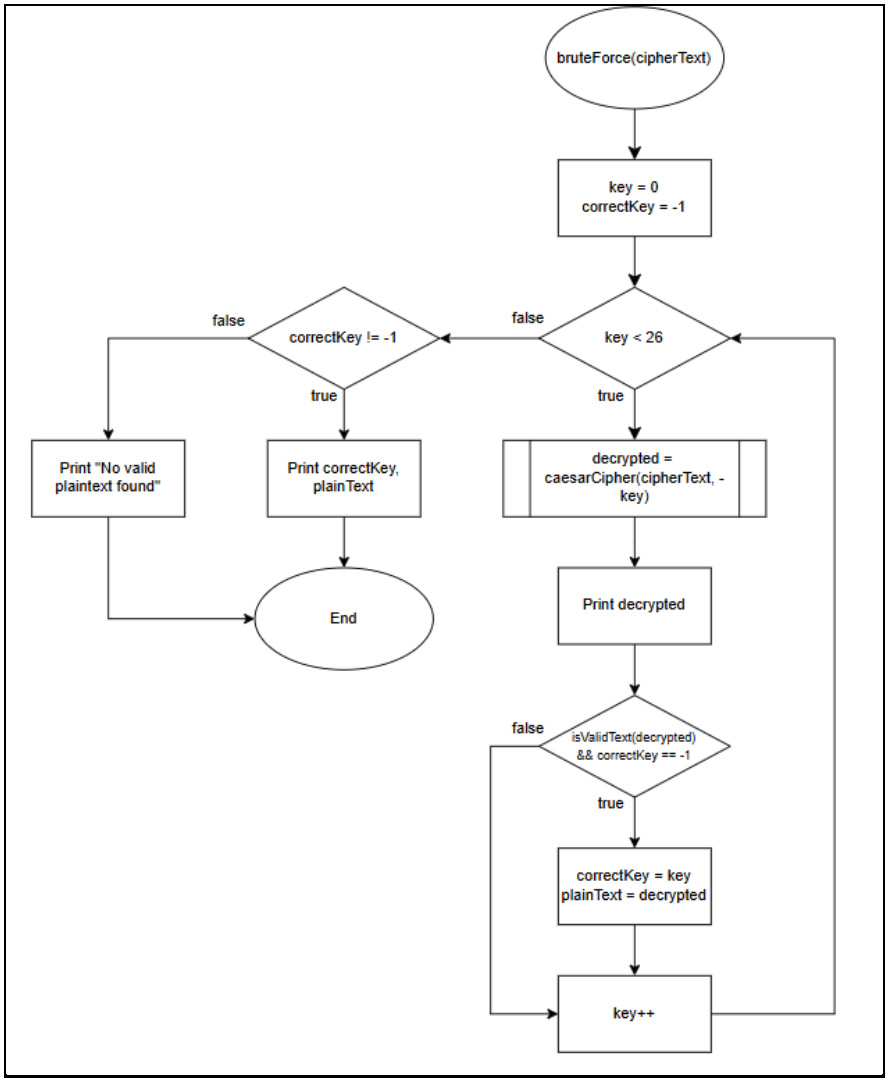
```
string caesarCipher(string text, int key)
{
   string result = text;

   for(int i = 0; i < text.length(); i++)
   {
      result[i] = shiftChar(text[i], key);
   }
```

```
    return result;
}
```

- Hàm shiftChar(c, key):
  - if(c >= ‘A’ && c <= ‘Z’): Kiểm tra ký tự có phải là chữ in hoa nằm trong khoảng từ A đến Z. Tương tự với if(c >= ‘a’ && c <= ‘z’) để kiểm tra là chữ thường.
  - return (c - ‘A’ + key + 26) % 26 + ‘A’:
    - c - ‘A’: chuyển ký tự hiện tại thành vị trí nằm trong bảng chữ cái (A = 0, B = 1,...).
    - c - ‘A’ + key: thực hiện dịch ký tự theo khóa k.
    - c - ‘A’ + key + 26: Tránh trường hợp khóa k là số âm khi giải mã làm cho kết quả âm.
    - (c - ‘A’ + key + 26) % 26: Giới hạn lại vị trí chữ cái nằm trong khoảng từ 0 - 25, tránh bị tràn khỏi số lượng chữ cái.
    - (c - ‘A’ + key + 26) % 26 + ‘A’: trả lại ký tự sau khi thực hiện mã hóa/giải mã theo khóa k.
  - return c: Trả lời các ký tự không phải là chữ cái (khoảng trắng, số, dấu cấu,...).

```
char shiftChar(char c, int key)
{
    if(c >= 'A' && c <= 'Z')
    {
        return (c - 'A' + key + 26) % 26 + 'A';
    }
    else if(c >= 'a' && c <= 'z')
    {
        return (c - 'a' + key + 26) % 26 + 'a';
    }

    return c;
}
```

- Hàm bruteForce(ciphertext):



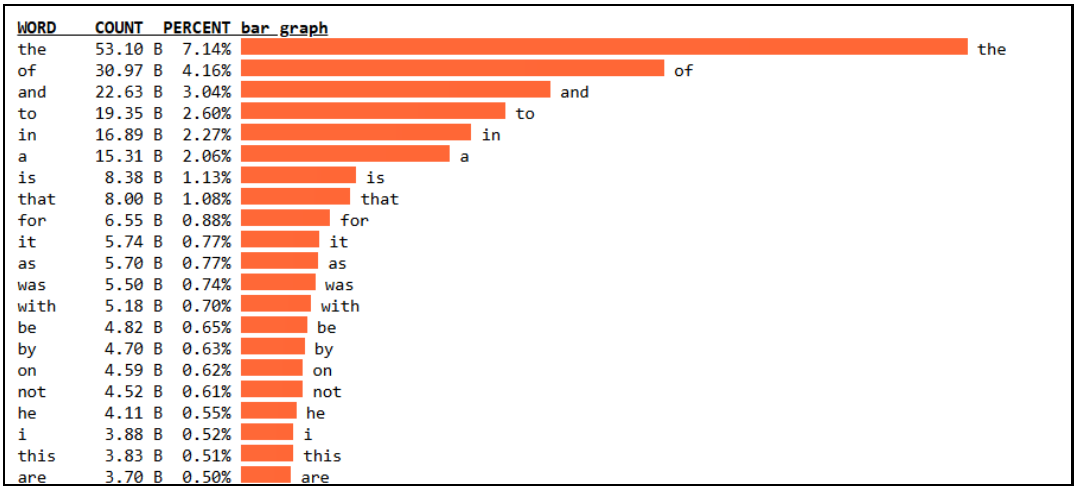
Hình 3. *Flowchart của hàm* ***bruteForce(cipherText)****.*

- Hàm này sẽ nhận đoạn CipherText. Thuật toán Brute-force sẽ thực hiện dò khóa k từ 1 đến 26 để tìm ra khóa k đã được dùng để mã hóa.
- correctKey = -1: giả định ban đầu chưa tìm được khóa đúng.
- Thực hiện chạy vòng lặp từ 0 đến 25.
- Gọi hàm **caesarCipher(cipherText, -key)** để thực giải mã ciphertext.
- Khi đã giải mã xong thì thực hiện in plaintext tương ứng với mỗi khóa ra màn hình.
- if(isValidPlaintext(decrypted) && correctKey == -1): hàm isValidPlaintext(decrypted) kiểm tra xem đoạn plaintext vừa được giải mã có phải là văn bản hợp lệ hay không. Điều kiện correctKey == -1 đảm bảo chỉ lưu kết quả đúng đầu tiên.
- Nếu đúng thì lưu lại khóa đúng và plaintext tương ứng.

- if(correctKey != -1): Sau khi vòng lặp kết thúc, thực hiện kiểm tra. Nếu tìm được key thì in ra Khóa k kèm với plaintext tương ứng. Nếu không tìm thấy thì in "No valid plaintext found.".

```
void bruteForce(string cipherText)
{
   int correctKey = -1;
   string correctPlaintext;

   for(int key = 0; key < 26; key++)
   {
      string decrypted = caesarCipher(cipherText, -key);

      cout << "Key " << key << " : " << decrypted << endl;

      if(isValidPlaintext(decrypted) && correctKey == -1)
      {
         correctKey = key;
         correctPlaintext = decrypted;
      }
   }

   if(correctKey != -1)
   {
      cout << "\nCorrect Key: " << correctKey << endl;
      cout << "Plaintext: " << correctPlaintext << endl;
   }
   else
   {
      cout << "\nNo valid plaintext found." << endl;
   }
}
```

- Hàm **isValidPlaintext(text)**:
  - Hàm này sẽ nhận đoạn plaintext và kiểm tra xem đoạn plaintext sau khi giải mã này có khả năng là Tiếng Anh hợp lệ hay không.
  - Dựa theo tần suất xuất hiện của các từ Tiếng Anh (theo **English Letter Frequency Counts: Mayzner Revisited or ETAOIN SRHLDCU**) để xác định đoạn plaintext trên có hợp lệ không.



WORD	COUNT	PERCENT
the	53.10 B	7.14%
of	30.97 B	4.16%
and	22.63 B	3.04%
to	19.35 B	2.60%
in	16.89 B	2.27%
a	15.31 B	2.06%
is	8.38 B	1.13%
that	8.00 B	1.08%
for	6.55 B	0.88%
it	5.74 B	0.77%
as	5.70 B	0.77%
was	5.50 B	0.74%
with	5.18 B	0.70%
be	4.82 B	0.65%
by	4.70 B	0.63%
on	4.59 B	0.62%
not	4.52 B	0.61%
he	4.11 B	0.55%
i	3.88 B	0.52%
this	3.83 B	0.51%
are	3.70 B	0.50%

***Hình 4**. Tần suất xuất hiện của các ký tự Tiếng Anh.*

- Không chọn các từ có 2 ký từ vì chúng có tần suất xuất hiện rất cao trong tiếng Anh, nên khi brute-force chúng dễ xuất hiện ngẫu nhiên trong các kết quả giải mã sai.
- Ví dụ 1 plaintext sai "xofktoinabc" vẫn có thể chứa "of", "to" và "in".
- Trong khi các từ dài hơn thì sẽ khó xuất hiện ngẫu nhiên, tăng cao độ tin cậy.
- Thêm dấu cách " the " thay vì "the" giúp đảm bảo từ đó là 1 từ độc lập.

```
bool isValidPlaintext(string text)
{
   if(text.find(" the ") != string::npos) return true;
   if(text.find(" and ") != string::npos) return true;
   if(text.find(" that ") != string::npos) return true;
   if(text.find(" for ") != string::npos) return true;
   if(text.find(" was ") != string::npos) return true;

   return false;
}
```

- Chạy thử Brute-Force với đoạn ciphertext "Max NBM bl t extwbgz bglmbmnmbhg ngwxk OGN-AVF, lixvbtebsbgz bg max ybxew hy bgyhkftmbhg mxvaghehzr." trong tài liệu hướng dẫn, ta thu được kết quả như sau:

```
PS D:\Coding\NT101\Lab01> cd "d:\Coding\NT101\Lab01\" ; if ($?) { g++ CaeserCipher.cpp -o CaeserCipher } ; if ($?) {
.\CaeserCipher }
===== Caesar Cipher =====
1. Encrypt
2. Decrypt
3. Brute Force
Choose: 3
Enter ciphertext: Max NBM bl t extwbgz bglmbmnmbhg ngwxk OGN-AVF, lixvbtebsbgz bg max ybxew hy bgyhkftmbhg mxvaghehzr
.
Key 0 : Max NBM bl t extwbgz bglmbmnmbhg ngwxk OGN-AVF, lixvbtebsbgz bg max ybxew hy bgyhkftmbhg mxvaghehzr.
Key 1 : Lzw MAL ak s dwsvafy afklalmlagf mfvwj NFM-ZUE, khwuasdarafy af lzw xawdv gx afxgjeslagf lwuzfgdgyq.
Key 2 : Kyv LZK zj r cvruzex zejkzklkzfe leuvi MEL-YTD, jgvtzrczqzex ze kyv wzvcu fw zewfidrkzfe kvtyefcfxp.
Key 3 : Jxu KYJ yi q buqtydw ydijyjkjyed kdtuh LDK-XSC, ifusyqbypydw yd jxu vyubt ev ydvehcqjyed jusxdebewo.
Key 4 : Iwt JXI xh p atpsxcv xchixijixdc jcstg KCJ-WRB, hetrxpaxoxcv xc iwt uxtas du xcudgbpixdc itrwcdadvn.
Key 5 : Hvs IWH wg o zsorwbu wbghwhihwcb ibrsf JBI-VQA, gdsqwozwnwbu wb hvs twszr ct wbtcfaohwcb hsqvbczcum.
Key 6 : Gur HVG vf n yrnqvat vafgvghgvba haqre IAH-UPZ, fcrpvnyvmvat va gur svryq bs vasbezngvba grpuabybtl.
Key 7 : Ftq GUF ue m xqmpuzs uzefufgfuaz gzpqd HZG-TOY, ebqoumxuluzs uz ftq ruqxp ar uzradymfuaz fqotzaxask.
Key 8 : Esp FTE td l wplotyr tydetefetzy fyopc GYF-SNX, dapntlwtktyr ty esp qtpwo zq tyqzcxletzy epnsyzwzrj.
Key 9 : Dro ESD sc k voknsxq sxcdsdedsyx exnob FXE-RMW, czomskvsjsxq sx dro psovn yp sxpybwkdsyx domrxyvyqi.
Key 10 : Cqn DRC rb j unjmrwp rwbcrcdcrxw dwmna EWD-QLV, bynlrjurirwp rw cqn ornum xo rwoxavjcrxw cnlqwxuxph.
Key 11 : Bpm CQB qa i tmilqvo qvabqbcbqwv cvlmz DVC-PKU, axmkqitqhqvo qv bpm nqmtl wn qvnwzuibqwv bmkpvwtwog.
Key 12 : Aol BPA pz h slhkpun puzapabapvu bukly CUB-OJT, zwljphspgpun pu aol mplsk vm pumvythapvu aljouvsvnf.
Key 13 : Znk AOZ oy g rkgjotm otyzozazout atjkx BTA-NIS, yvkiogrofotm ot znk lokrj ul otluxsgzout zkinturume.
Key 14 : Ymj ZNY nx f qjfinsl nsxynyzynts zsijw ASZ-MHR, xujhnfqnensl ns ymj knjqi tk nsktwrfynts yjhmstqtld.
Key 15 : Xli YMX mw e piehmrk mrwxmxyxmsr yrhiv ZRY-LGQ, wtigmepmdmrk mr xli jmiph sj mrjsvqexmsr xiglrspskc.
Key 16 : Wkh XLW lv d ohdglqj lqvwlwxwlrq xqghu YQX-KFP, vshfldolclqj lq wkh ilhog ri lqirupdwlrq whfkqrorjb.
Key 17 : Vjg WKV ku c ngcfkpi kpuvkvwvkqp wpfgt XPW-JEO, urgekcnkbkpi kp vjg hkgnf qh kphqtocvkqp vgejpqnqia.
Key 18 : Uif VJU it b mfbejoh jotujuvujpo voefs WOV-IDN, tqfdjbmjajoh io uif gjfme pg jogpsnbujpo ufdiopmphz.
Key 19 : The UIT is a leading institution under VNU-HCM, specializing in the field of information technology.
Key 20 : Sgd THS hr z kdzchmf hmrshstshnm tmcdq UMT-GBL, rodbhzkhyhmf hm sgd ehdkc ne hmenqlzshnm sdbgmnknfx.
Key 21 : Rfc SGR gq y jcybgle glqrgrsrgml slbcp TLS-FAK, qncagyjgxgle gl rfc dgcjb md gldmpkyrgml rcaflmjmew.
Key 22 : Qeb RFQ fp x ibxafkd fkpqfqrqflk rkabo SKR-EZJ, pmbzfxifwfkd fk qeb cfbia lc fkclojxqflk qbzeklildv.
Key 23 : Pda QEP eo w hawzejc ejopepqpekj qjzan RJQ-DYI, olayewhevejc ej pda beahz kb ejbkniwpekj paydjkhkcu.
Key 24 : Ocz PDO dn v gzvydib dinodopodji piyzm QIP-CXH, nkzxdvgdudib di ocz adzgy ja diajmhvodji ozxcijgjbt.
Key 25 : Nby OCN cm u fyuxcha chmncnoncih ohxyl PHO-BWG, mjywcufctcha ch nby zcyfx iz chzilguncih nywbhifias.

Correct Key: 19
Plaintext: The UIT is a leading institution under VNU-HCM, specializing in the field of information technology.
PS D:\Coding\NT101\Lab01>
```

***Hình 5**. Kết quả nhận được sau khi thực hiện chức năng Brute-Force.*

Task 2.2: Mono-alphabetic Substitution Cipher and Frequency Analysis

Bạn được cung cấp một tệp ciphertext (2.2_cipher_text) (https://pastecode.io/s/h3hvp47p) đã được mã hóa bằng mono-alphabetic substitution cipher. Nhiệm vụ là khôi phục lại bản rõ bằng phương pháp frequency analysis và mô tả chi tiết quy trình thực hiện theo từng bước, bao gồm cách thống kê tần suất, cách đưa ra giả thuyết về các ký tự, và cách kiểm tra, điều chỉnh cho đến khi thu được văn bản có nghĩa. Biết trước rằng bản rõ là một bài báo tiếng Anh.

V	488×	10.66%
R	403×	8.8%
M	397×	8.67%
G	383×	8.37%
L	365×	7.97%
Z	339×	7.4%
S	339×	7.4%
I	289×	6.31%
O	208×	4.54%
X	190×	4.15%
H	188×	4.11%
W	169×	3.69%
F	141×	3.08%

***Hình 5**. Bảng giá trị tần suất xuất hiện của các ký tự đơn.*

- Dựa trên English Letter Frequency Counts (Mayzner Revisited) hoặc quy tắc tần suất ký tự ETAOIN SHRDLU, ký tự "E" thường là ký tự xuất hiện với tần suất cao nhất trong tiếng Anh. Do đó, khi quan sát bản mã, ta giả định rằng ký tự "V" tương ứng với ký tự "E" trong bản rõ.

↑↓	↑↓
LMW	21
RZM	16
GHV	12
RMT	10
GVI	10
IVS	10
VIS	9
VLI	9
RVS	9
VMG	8
VIR	7
SLM	7

***Hình 6**. Bảng giá trị tần suất xuất hiện của chuỗi 3 ký tự.*

- Khi phân tích các ký tự xuất hiện đơn lẻ, nhận thấy "L" thường đứng một mình. Trong tiếng Anh, chỉ có "A" và "I" có thể xuất hiện độc lập, trong đó "A" phổ biến hơn, nên giả định "L" → "A".

- Từ đó, cụm “LMW” xuất hiện nhiều lần có thể tương ứng với “AND”, là cụm ba ký tự phổ biến bắt đầu bằng “A”.
- Xét giả thuyết dùng quy luật Atbash cipher, ký tự “Z” sẽ giải mã thành “A”. Tuy nhiên, cụm “zmv” -> “ANE” không có nghĩa, nên loại bỏ giả thuyết này và tạm giả định “Z” -> “O”.
- Với cụm “RS”, theo Atbash thì “R” -> “I”, suy ra “RS” -> “IS”, là từ phổ biến. Do đó, “S” -> “S” là một ngoại lệ.
- Cuối cùng, dựa trên tính đối xứng của Atbash (L <-> O) và đã biết “L” -> “A”, suy ra thêm “O” -> “L”.

Mã hóa	Gốc	Mã hóa	Gốc
A	X	N	M
B	Y	O	L
C	Z	P	K
D	F	Q	J
E	V	R	I
F	U	S	S
G	T	T	G
H	H	U	W
I	R	V	E
J	Q	W	D
K	P	X	C
L	A	Y	B
M	N	Z	O

- Áp dụng quy tắc mã hóa trên, ta được đoạn thông điệp:

The Harry Potter series, written by British author J.K. Rowling, is one of the most popular and influential fantasy sagas of modern literature. Spanning seven books, the series follows the life of a young boy, Harry Potter, who discovers on his 11th birthday that he is a wizard. He is invited to attend Hogwarts School of Witchcraft and Wizardry, where he learns about his magical heritage, develops close friendships, and confronts the dark forces that threaten the wizarding world. The central plot revolves around Harry's ongoing battle with Lord Voldemort, a dark wizard intent on conxuering both the magical and non-magical worlds. As Harry grows older, he learns more about his own mysterious past, his connection to Voldemort, and the prophecy that ties their fates together. Throughout the series, themes of friendship, loyalty, bravery, and the power of love are eqplored, with a rich world filled with magical creatures, spells, and deep mythology. The books, starting with Harry Potter and the Philosopher's Stone (1997) and concluding with *Harry Potter and the Deathly Hallows (2007), have sold over 500 million copies worldwide, been translated into more than 80 languages, and adapted into a successful film

series. The series is known not only for its enchanting world-building but also for addressing darker, compleq themes such as mortality, prejudice, and the abuse of power. The Harry Potter series has left an indelible mark on popular culture, fostering a global fan base, inspiring countless spin-offs, and cementing its place as a literary phenomenon. Vietnam National University, Ho Chi Minh City (VNU-HCM) is one of the most prestigious and comprehensive higher education institutions in Vietnam, known for its excellence in research, innovation, and education. Founded in 1995 by the Vietnamese government, VNU-HCM was established as a key center for training, research, and technology transfer with the mission of contributing to the countryΓÇÖs socio-economic development. As a flagship university, it has played a pivotal role in producing highly skilled professionals, scientists, and leaders across multiple fields. VNU-HCM operates under a distinctive structure as a multi-disciplinary and multi-campus system, comprising several member universities and research institutes. These include some of the most well-established institutions in the region, such as the University of Science, University of Technology, University of Social Sciences and Humanities, International University, University of Economics and Law, and others. Each of these member institutions operates semi-independently but collaborates under the umbrella of VNU-HCM, making it a unique higher education model in Vietnam that promotes interdisciplinary learning and research. Located in Ho Chi Minh City, the country's economic and cultural hub, VNU-HCM benefits from close ties with industries, businesses, and government agencies. This proximity fosters collaboration in research, internships, and job placement, ensuring that its graduates are well-prepared to meet the demands of an evolving workforce. The university is committed to producing graduates who are not only academically proficient but also equipped with practical skills and a global outlook. VNU-HCM is renowned for its strong emphasis on scientific research and technological innovation. It houses numerous research centers, institutes, and labs that focus on various areas of science, engineering, social sciences, economics, and more. The university's research agenda is aligned with national priorities, focusing on areas such as information technology, biotechnology, renewable energy, and environmental sustainability. VNU-HCM also actively engages in international cooperation, partnering with leading universities and research institutions worldwide to enhance its academic programs, foster joint research projects, and promote student and faculty exchanges. The university is also deeply involved in addressing global challenges through its research and educational programs. VNU-HCM places a strong emphasis on sustainable development, environmental conservation, and technological innovation, aligning its goals with the United Nations Sustainable Development Goals (SDGs). The institution encourages its faculty and students to pursue research that has a meaningful impact on both the

local and global community. In terms of student life, VNU-HCM offers a vibrant and dynamic environment. With a student body of over 60,000 and a faculty of highly qualified professors and researchers, the university provides a rich academic and social experience. The campuses are equipped with modern facilities, including libraries, laboratories, sports complexes, and dormitories, creating a supportive learning environment. Students also have access to a variety of extracurricular activities, clubs, and organizations, allowing them to engage in cultural, social, and professional development alongside their studies. In conclusion, Vietnam National University - Ho Chi Minh City stands as a leading educational institution in Vietnam, contributing significantly to the country intellectual, scientific, and economic progress. Its commitment to academic excellence, research innovation, and societal impact has positioned it as a crucial player in shaping the future of Vietnam and the broader region. As it continues to evolve and expand, VNU-HCM remains dedicated to fostering the next generation of leaders and innovators who will drive sustainable growth and global progress.

Task 2.3 Mono-alphabetic Substitution Cipher

Bạn hãy xây dựng một chương trình hỗ trợ giải mã tự động mono-alphabetic substitution cipher khi chỉ biết ciphertext. Chương trình cần cho phép nhập một ciphertext bất kỳ hoặc đọc từ file và thực hiện giải mã tự động để tìm ra bản rõ có khả năng đúng cao nhất dựa trên các đặc trưng thống kê của ngôn ngữ. Kết quả cần hiển thị bản rõ tốt nhất tìm được và thông tin cơ bản về quá trình thực hiện. Bạn cần kiểm thử chương trình với ciphertext ở Task 3.1 hoặc các văn bản khác và nhận xét về hiệu quả của phương pháp.

Phân tích vấn đề:

- Trong mã hóa thay thế, mỗi chữ cái trong bảng chữ cái được thay thế bằng một chữ cái khác. Tuy nhiên, các đặc trưng thống kê của ngôn ngữ (ví dụ như tiếng Anh) vẫn được bảo toàn:
  - Tần suất chữ cái đơn: ‘E’, ‘T’, ‘A’ xuất hiện nhiều nhất
  - Tần suất cặp chữ (Bigrams): ‘TH’, ‘HE’, ‘IN’ xuất hiện thường xuyên

Ý tưởng thuật toán:

1. Khởi tạo: Tạo một khóa ngẫu nhiên (hoặc dựa trên tần suất chữ cái đơn).
2. Đánh giá: Giải mã ciphertext bằng khóa này và tính điểm dựa trên log-probability của các n-grams (thường dùng Quadgrams - bộ 4 chữ cái) so với dữ liệu tiếng Anh chuẩn.
3. Xáo trộn (Hill climbing): Hoán đổi ngẫu nhiên 2 vị trí trong khóa để tạo khóa mới

4. Lựa chọn: Nếu khóa mới cho điểm cao hơn (văn bản trông giống tiếng Anh hơn) thì ta giữ lại khóa mới.
5. Lặp lại: Thực hiện hàng nghìn lần cho đến khi điểm số không tăng thêm được nữa.

Thiết kế chương trình

Bước 1: Chuẩn hóa dữ liệu (Preprocessing)

Lọc ra những kí tự đặc biệt, số, dấu cách, dấu câu,.. để đảm bảo máy tính chỉ xử lý 26 chữ cái tiếng Anh thuần túy. Sau đó toàn bộ chuỗi sẽ được chuẩn hóa về chữ in hoa.

- Sử dụng vòng lặp duyệt qua từng ký tự của chuỗi nhập vào.
- Hàm isalpha(c): Kiểm tra xem ký tự có phải là chữ cái không (loại bỏ số, dấu cách, dấu câu).
- toupper(c): Chuyển tất cả về chữ cái in hoa.
- Biến result để lưu chuỗi đã được xử lý.

```
// Hàm loại bỏ những kí tự không phải là chữ cái và chuển tất cả thành chữ hoa
string cleanText(string text)
{
    string result;

    for (char c : text)
    {
        if (isalpha(c))
            result += toupper(c);
    }
    return result;
}
```

Mục đích: Để khi tính toán cặp chữ cái (Bigram) ở những bước sau, chương trình không bị lỗi khi gặp các ký tự lạ.

Bước 2: Khởi tạo trạng thái ban đầu (Initialization)

Bước khởi tạo này nhằm tạo một trạng thái ban đầu cho quá trình tối ưu.Tạo ra một khóa ngẫu nhiên được sinh ra bằng cách hoán vị bảng chữ cái. Khóa này được sử dụng để giải mã ciphertext, tạo ra một bản plaintext tạm thời. Sau đó, bản plaintext này được đánh giá dựa trên tần suất xuất hiện của các bigram trong tiếng Anh.

- Hàm randomKey(): Trộn ngẫu nhiên bảng chữ cái ABCDEFGHIJKLMNOPQRSTUVWXYZ" bằng random_shuffle() để tạo ra khóa đầu tiên

```
// /hàm tạo key ngẫu nhiên
string randomKey()
{
    string key = alphabet;
    random_shuffle(key.begin(), key.end());
    return key;
}
```

- Hàm decrypt(): Dùng chìa khóa ngẫu nhiên đó để giải mã thử Ciphertext bằng cách:
  - Copy một chuỗi plain giống cipher.
  - Duyệt từng chữ trong cipher text.
  - Xác định vị trí chữ cái.
  - Thay thế bằng key .
  - Trả về plaintext sau khi giải mã.

```
// Hàm giải mã bằng cách sử dụng key
string decrypt(string cipher, string key)
{
    string plain = cipher;

    for (int i = 0; i < cipher.size(); i++)
    {
        int pos = cipher[i] - 'A';
        plain[i] = key[pos];
    }

    return plain;
}
```

- Hàm scoreText dùng để đánh giá mức độ "giống tiếng Anh" của văn bản giải mã.
- Tính điểm cho văn bản đã được giải mã thử này dựa trên tần suất của các bigram.
- Chấm điểm xem nó có giống tiếng Anh hay không nếu chuỗi có nhiều bigram phổ biến thì điểm cao và ngược lại.
  - Tạo một biến point để lưu tổng điểm.
  - Duyệt qua từng cặp ký tự liên tiếp (bigram) trong chuỗi và cộng điểm dựa trên tần suất xuất hiện của các bigram trong tiếng Anh.
  - Các bigram phổ biến sẽ được cộng điểm cao hơn (ít âm hơn), trong khi các bigram hiếm hoặc không tồn tại sẽ bị phạt điểm.

- Tổng điểm càng cao (gần 0) thì văn bản càng có khả năng là tiếng Anh hợp lệ.

```
// tính điểm cho văn bản giải mã dựa trên tần suất của các bigram
double scoreText(string text)
{
    double point = 0;

    for (int i = 0; i < text.size() - 1; i++){
        string bg = text.substr(i, 2);

        if (bigram.count(bg))
            point += bigram[bg];
        else
            point -= 3;
    }
    return point;
}
```

Mục đích: Tạo ra một vị trí ban đầu trên bản đồ điểm số trước khi bắt đầu leo núi.

Bước 3: Vòng lặp tối ưu hóa (Hill Climbing Loop)

Bước Hill Climbing nhằm cải thiện dần khóa giải mã bằng cách thực hiện các thay đổi nhỏ (hoán đổi hai ký tự trong khóa). Tại mỗi bước, thuật toán tạo ra một khóa mới, giải mã văn bản và tính điểm đánh giá. Nếu khóa mới cho kết quả tốt hơn (điểm cao hơn), nó sẽ được chấp nhận; ngược lại sẽ bị loại bỏ. Quá trình này được lặp lại nhiều lần để tiến dần đến một khóa tối ưu, giúp bản giải mã ngày càng giống với tiếng Anh.

Logic code:

- Chọn 2 vị trí ngẫu nhiên trong khóa hiện tại và đổi chỗ chúng.
- Giải mã với key mới.
- Chấm điểm bản giải mã mới. Nếu điểm cao hơn bản cũ, chúng ta sẽ giữ lại chìa khóa này. Nếu thấp hơn, nó vứt đi và thử hoán đổi khác.

```
// hoán đổi 2 vị trí trong key
string swapKey(string key)
{
    int a = rand() % 26;
    int b = rand() % 26;

    swap(key[a], key[b]);

    return key;
}

// Hàm thực hiện thuật toán Hill Climbing để tìm key tốt nhất
string hillClimb(string cipher, double& bestScore)
{
    string key = randomKey();
    string bestKey = key;

    string plain = decrypt(cipher, key);
    bestScore = scoreText(plain);

    for (int i = 0; i < 5000; i++)
    {
        string newKey = swapKey(key);

        string newPlain = decrypt(cipher, newKey);
        double newScore = scoreText(newPlain);

        if (newScore > bestScore)        {
            bestScore = newScore;
            key = newKey;
            bestKey = newKey;
        }
    }

    return bestKey;
}
```

Mục đích: Cải thiện dần dần chìa khóa để bản giải mã ngày càng giống tiếng Anh thật.

Bước 4: Tránh bẫy kết quả sai (Restarting Strategy)

Do thuật toán Hill Climbing có thể bị mắc kẹt tại các điểm tối ưu cục bộ (local optimum), chương trình sử dụng chiến lược Random Restart. Cụ thể, thuật toán được chạy nhiều lần độc lập với các khóa khởi tạo ngẫu nhiên khác nhau. Mỗi lần chạy sẽ tạo ra một kết quả riêng biệt. Sau đó, chương trình lựa chọn kết quả có điểm số cao nhất trong tất cả các lần chạy, từ đó tăng khả năng tìm được lời giải tối ưu toàn cục.

Logic code:

- srand(time(0)): Giúp mỗi lần chạy chương trình thì random kết quả khác nhau

- Nhập ciphertext.
- Khai báo biến kết quả.
- Mỗi vòng lặp là một lần tạo lại từ đầu.

```
int main()
{
    srand(time(0));

    string cipher;

    cout << "Nhap ciphertext:\n";
    getline(cin, cipher);

    cipher = cleanText(cipher);

    string bestKey;
    string bestPlain;

    double globalBestScore = -1e9;

    int restarts = 100;

    for (int i = 0; i < restarts; i++)
    {
        double score;

        string key = hillClimb(cipher, score);

        if (score > globalBestScore)
        {
            globalBestScore = score;
            bestKey = key;
        }
    }

    bestPlain = decrypt(cipher, bestKey);
```

Bước 5: Xuất kết quả cuối cùng (Output)

Sau khi đã chạy đủ số lần thử nghiệm, chương trình chọn ra khóa tốt nhất.

Logic code:

- Hàm decrypt: chọn ra khóa có điểm số cao nhất vcagiải mã văn bản bằng chìa khóa đó.
- In ra màn hình khóa và đoạn văn bản đã được dịch.

```
    bestPlain = decrypt(cipher, bestKey);

    cout << "KET QUA TOT NHAT" << endl;

    cout << "Key: " << bestKey << endl;

    cout << "\nPlaintext:\n" << bestPlain << endl;

    return 0;
}
```

Kết quả kiểm thử

Chương trình được kiểm thử với một văn bản dài hơn 100 từ. Sau khi mã hóa và giải mã lại bằng cùng khóa, kết quả thu được văn bản đã đọc khoảng **80-85%** nội dung chính. Các từ phổ biến (THE, AND, IS) đã được giải mã chính xác, ngoài ra một số ký tự hiếm hoặc ít xuất hiện trong danh sách Bigram (như Y, W) bị giải mã nhầm sang B, Z.

Key tìm được: JBPYLGTHRVQANWCXFISUZEDMKO

Plaintext thu được:

```
Nhap ciphertext:
Ghv Hliib Kzggvi svirvs, uirggvm yb Yirgrsh lfghzi Q.P. Izuormt, rs zmv zd ghv nzsg kzkfoli lmw rmdofvmgrlo dlmglsb sltl
s zd nzwvim orgvilgfiv. Sklmmrmt svevm yzzps, ghv svirvs dzoozus ghv ordv zd l bzfmt yzb, Hliib Kzggvi, uhz wrsxzevis zm
 hrs 11gh yrighwlb ghlg hv rs l urcliw. Hv rs rmergvw gz lggvmw Hztuligs Sxhzzo zd Urgxhxildg lmw Urcliwib, uhviv hv ovl
ims lyzfg hrs nltrxlo hvirgltv, wvevozks xozsv dirvmwshrks, lmw xzmdizmgs ghv wlip dzixvs ghlg ghivlgvm ghv urcliwrmt uz
iow.
KET QUA TOT NHAT
Key: JBPYLGTHRVQANWCXFISUZEDMKO
Score: -1032.2

Plaintext:
THEHARRBQOTTERSERIESZRITTENKBKRITISHAGTHORFXROZCINUISONEOYTHEWOSTQOQGCARANDINYCGENTIACYANTASBSAUASOYWODERNCITERATGRESQAN
NINUSELENKOOXSTHESERIESYOCCOZSTHECIYEOYABOGNUKOBHARRBQOTTERZHODISMOLERSONHISTHKIRTHDABTHATHEISAZIPARDHEISINLITEDTOATTEND
HOUZARTSSMHOOCOYZITMHMRAYTANDZIPARDRBZHEREHECEARNSAKOGTHISWAUIMACHERITAUEDELECOQSMCOSEYRIENDSHIQSANDMONYRONTSTHEDARXYORM
ESTHATTHREATENTHEZIPARDINUZORCD
```

Task 2.4. Playfair Cipher

Yêu cầu

Tìm hiểu thuật toán mã hóa và giải mã Playfair Cipher và viết một chương trình để mã hóa và giải mã bằng Playfair cipher. Cho phép nhập khóa và bản rõ để mã hóa hoặc nhập khóa và ciphertext để giải mã. (Lưu ý: Chương trình phải hiển thị ma trận Playfair 5×5 tương ứng với khóa đã nhập)

Phân tích vấn đề

Chia bản rõ thành từng cặp. Nếu cặp có 2 chữ cái giống nhau (ví dụ: "EE"), chèn một ký tự giả (thường là 'X') vào giữa. Nếu tổng số ký tự lẻ, thêm 'X' vào cuối.

- Cùng hàng: Thay mỗ i chữ cái bằng chữ cái bên phải nó (cuốn chiếu vòng tròn).
- Cùng cột: Thay mỗi chữ cái bằng chữ cái bên dưới nó (cuốn chiếu vòng tròn).

- Tạo thành hình chữ nhật: Thay mỗi chữ cái bằng chữ cái nằm cùng hàng với nó nhưng ở cột của chữ cái kia trong cặp.

Ý tưởng thuật toán

1. Khởi tạo ma trận

- Hợp nhất ký tự: Vì bảng chữ cái tiếng Anh có 26 chữ, gom I và J là một.
- Ưu tiên từ khóa: Từ khóa (Key) được điền vào trước để tạo sự xáo trộn. Các chữ cái lặp lại trong từ khóa chỉ được viết một lần
- Lấp đầy khoảng trống: Những ô còn lại sẽ được điền bằng các chữ cái chưa xuất hiện trong từ khóa theo thứ tự từ A đến Z.

2. Xử lý dữ liệu đầu vào

- Tách cặp: Chia bản rõ thành các cặp 2 chữ cái
- Chèn ký tự đệm X: Nếu gặp cặp chữ cái giống nhau,chèn 'X' vào giữa. Ngoài ra, nếu sau khi tách mà cuối cùng thừa ra 1 chữ cái đơn lẻ, ta thêm 'X' vào cuối để đủ cặp

3. Quy tắc mã hóa

- Quy tắc 1: Cùng hàng
    - Thay mỗi chữ cái bằng chữ cái nằm ngay bên phải nó. Nếu ở cuối hàng thì quay lại đầu hàng
- Quy tắc 2: Cùng cột
    - Thay mỗi chữ cái bằng chữ cái nằm ngay dưới nó. Nếu ở cuối cột thì quay lên đầu cột
- Quy tắc 3: Tạo thành hình chữ nhật
    - Đây là trường hợp phổ biến nhất. Xác định hình chữ nhật có 2 góc là A và B. Chữ cái thay thế cho A sẽ là chữ cái nằm cùng hàng với A nhưng ở cột của B. Tương tự làm như vậy với B.

4. Ý tưởng Giải mã

- Cùng hàng: Dịch sang trái (thay vì sang phải).
- Cùng cột: Dịch lên trên (thay vì xuống dưới).
- Hình chữ nhật: Giữ nguyên quy tắc tráo đổi góc (vì hình chữ nhật không đổi)

Thiết kế chương trình

Bước 1: Chuẩn hóa dữ liệu (Preprocessing)

Mục đích: Đảm bảo dữ liệu đầu vào chỉ gồm các chữ cái hợp lệ để tránh lỗi khi mã hóa và giải mã.

Logic code:

- Duyệt từng ký tự trong chuỗi
- isalpha(c): kiểm tra có phải chữ cái
- toupper(c): chuyển sang chữ hoa
- Nếu c == 'J' thì đổi thành I
- Lưu vào chuỗi res

```
// Hàm xử lý chuỗi: Viết hoa, đổi J thành I, loại bỏ ký tự không phải chữ cái
string cleanString(string s){
    string res = "";
    for (char c : s){
        if (isalpha(c)){
            c = toupper(c);
            if (c == 'J') c = 'I';
            res += c;
        }
    }
    return res;
}
```

Bước 2: Tạo ma trận Playfair 5 x 5

Mục đích: Tạo bảng 5 x 5 chứa 25 chữ cái không trùng để phục vụ mã hóa

Thực hiện:

- Chuẩn hóa khóa
- Ghép với bảng chữ cái
- Loại bỏ ký tự trùng
- Điền vào ma trận 5 x 5

Logic code:

- Tạo chuỗi
- Duyệt từng ký tự để đảm bảo không trùng
- Điền vào ma trận

```
// Tạo ma trận Playfair 5x5
void generateMatrix(string key, char matrix[5][5]){
    string alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ";
    string combined = cleanString(key) + alphabet;
    string finalKey = "";

    for (char c : combined){
        if (finalKey.find(c) == string::npos){
            finalKey += c;
        }
    }

    int k = 0;
    for (int i = 0; i < 5; i++){
        for (int j = 0; j < 5; j++){
            matrix[i][j] = finalKey[k++];
        }
    }
}
```

Bước 3: Hiển thị ma trận

Mục đích: Kiểm tra trực quan xem ma trận đúng hay chưa

Logic code:

- Duyệt ma trận 2 chiều
- In từng phần tử

```
// Hiển thị ma trận
void displayMatrix(char matrix[5][5]){
    cout << "\nMa tran Playfair 5x5\n";
    for (int i = 0; i < 5; i++){
        for (int j = 0; j < 5; j++){
            cout << matrix[i][j] << "  ";
        }
        cout << endl;
    }
}
```

Bước 4: Tìm vị trí ký tự trong ma trận

Mục đích: Xác định tọa độ hàng, cột của mỗi ký tự để phục vụ mã hóa và giải mã.

Logic code:

- Duyệt toàn bộ ma trận
- Nếu tìm thấy ký tự thì trả về tọa độ hàng, cột

```
// Tìm vị trí của ký tự trong ma trận
Position findPos(char matrix[5][5], char c){
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (matrix[i][j] == c) return { i, j };
        }
    }
    return { -1, -1 };
}
```

Bước 5: Chuẩn hóa bản rõ thành cặp

Mục đích: Chuyển plaintext thành các cặp ký tự theo quy tắc Playfair

Quy tắc:

- Nếu 2 chữ giống nhau thì chèn 'X'
- Nếu chuỗi lẻ thì thêm 'X' vào cuối

Logic code:

- Duyệt từng ký tự
- Nếu 2 chữ cái giống nhau trong 1 cặp thì chèn X
- Thêm X nếu chuỗi lẻ

```
// Xử lý bản rõ thành các cặp (digraphs)
string prepareText(string text){
    text = cleanString(text);
    string prepared = "";
    for (int i = 0; i < text.length(); i++){
        prepared += text[i];
        if (i + 1 < text.length()){
            if (text[i] == text[i + 1]){
                prepared += 'X'; // Chèn X nếu 2 chữ cái giống nhau trong 1 cặp
            }
            else{
                prepared += text[++i];
            }
        }
        else{
            prepared += 'X'; // Thêm X nếu chuỗi lẻ
        }
    }
    return prepared;
}
```

Bước 6: Mã hóa và giải mã

Mục đích: Thực hiện thuật toán Playfair để mã hóa hoặc giải mã.

Logic code:

- Nếu tham số direction bằng 1 thì mã hóa, còn direction bằng -1 thì giải mã
- Quy tắc xử lý
- Cùng hàng
    - Mã hóa: dịch sang phải
    - Giải mã: dịch sang trái
- Cùng cột
    - Mã hóa: dịch xuống
    - Giải mã: dịch lên
- Khác hàng và cột (Hình chữ nhật)
    - Hoán đổi cột của 2 ký tự

```
// Hàm xử lý mã hóa/giải mã
string process(string text, char matrix[5][5], int direction){
    string result = "";
    for (int i = 0; i < text.length(); i += 2) {
        Position p1 = findPos(matrix, text[i]);
        Position p2 = findPos(matrix, text[i + 1]);

        if (p1.row == p2.row){ // Cùng hàng
            result += matrix[p1.row][(p1.col + direction + 5) % 5];
            result += matrix[p2.row][(p2.col + direction + 5) % 5];
        }
        else if (p1.col == p2.col){ // Cùng cột
            result += matrix[(p1.row + direction + 5) % 5][p1.col];
            result += matrix[(p2.row + direction + 5) % 5][p2.col];
        }
        else{ // Khác hàng và cột
            result += matrix[p1.row][p2.col];
            result += matrix[p2.row][p1.col];
        }
    }
    return result;
}
```

Kết quả kiểm thử

Chương trình được thực hiện kiểm thử với các đoạn văn bản dài (hơn 100 từ) để kiểm tra khả năng xử lý chuỗi và độ ổn định của ma trận.

- Key sử dụng: Harry Potter
- Văn bản mẫu:
  ARYWYPHCBVEBYGMPNCYGCNTDNCWTMGRMFTQPLEWTMLREFBEBQEBIYG
  BFLPHVOAEHKDHEUNGQFEROLEWTMLOPHEQGOSBEROQDWTLCMTHBWLN
  RKXRYLORYYPHCBVEBYRLGYDMKYGGWKLROANDBWGNERMNGYRLGHEWRT
  RLMBRHMUDGVODVTEGMCHLGWCMTFODNRRYCMZKODDUTDXGEOPOYRMF

RMGUKXRYGHABROVTGQMCEHPRPEOTSEGEQLARYWYPOTMGQDOEXGOAUD
HGUTULTNEHFTFHPGXGVPHGURBDMEGWKLETCBOTNTFQLTAEHMTUGEOA
HEVEROXGVPHGDEWTEWGQIEDLPILERWPMOATNGQKQEAHBMVRFKBRMKL
XODXFREBHMNUKXRYKLRMFLWDDNCN

Quá trình thực thi:

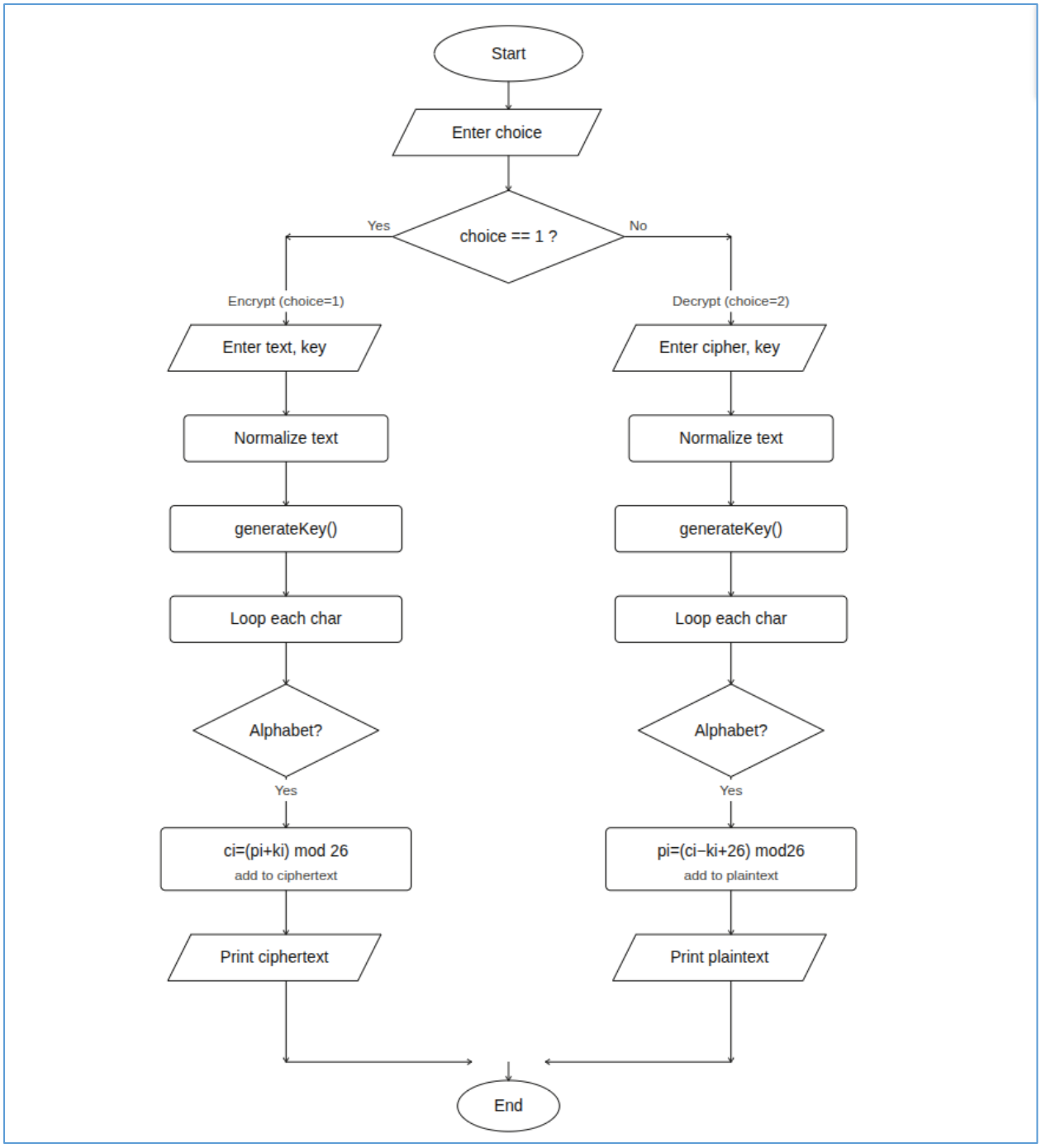
- Khởi tạo ma trận: Từ khóa Harry Potter được xử lý thành HARYPOTE và điền vào ma trận 5 x 5.
- Chọn thao tác (mã hóa hoặc giải mã)

  Đánh giá kết quả:

- Độ chính xác: Văn bản sau khi giải mã đạt độ chính xác 100% so với văn bản đã qua tiền xử lý. Các cụm từ quan trọng như BRITISH AUTHOR, J.K. ROWLING, WIZARD được phục hồi nguyên vẹn.
- Chương trình hoạt động ổn định, xử lý tốt các trường hợp chữ cái lặp lại đứng cạnh nhau và các văn bản có độ dài lớn. Ma trận Playfair hiển thị rõ ràng, giúp người dùng dễ dàng đối chiếu quy tắc mã hóa thủ công.

```
Nhap khoa (Key): Harry Potter

Ma tran Playfair 5x5
H  A  R  Y  P
O  T  E  B  C
D  F  G  I  K
L  M  N  Q  S
U  V  W  X  Z
Chon thao tac (1. Ma hoa / 2. Giai ma): 2
Nhap ban ma (Ciphertext): ARYWYPHCBVEBYGMPNCYGCNTDNCWTMGRMFTQPLEWTMLREFBEBQEBIYGBFLPHVOAEHKDHEUNGQFEROLEWTMLOPHEQGOSBERO
QDWTLCMTHBWLNRKXRYLORYYPHCBVEBYRLGYDMKYGGWKLROANDBWGNERMNGYRLGHEWRTRLMBRHMUDGVODVTEGMCHLGWCMTFODNRRYCMZKODDUTDXGEOPOYRMF
RMGUKXRYGHABROVTGQMCEHPRPEOTSEGEQLARYWYPOTMGQDOEXGOAUDHGUTULTNEHFTFHPGXGVPHGURBDMEGWKLETCBOTNTFQLTAEHMTUGEOAHEVEROXGVPHG
DEWTEWGQIEDLPILERWPMOATNGQKQEAHBMVRFKBRMKLXODXFREBHMNUKXRYKLRMFLWDDNCN
Ket qua Giai ma: HARXRYPOTXTERISASERIESOFSEVENFANTASYNOVELSWRITTENBYBRITISHAUTHORIKROWLINGTHENOVELSCHRONICLETHELIVESOFAY
OUNGWIZARDHARRYPOTXTERANDHISFRIENDSHERMIONEGRANGERANDRONWEASLEYALLOFWHOMARESTUDENTSATHOGWARTSSCHOOLOFWITCHCRAFTANDWIZARD
RYTHEMAINSTORYARCCONCERNSHARXRYCONFLICTWITHLORDVOLDEMORTADARKWIZARDWHOINTENDSTOBECOMEIMMORTALOVERTHROWTHEWIZARDGOVERNING
BODYKNOWNASTHEMINISTRYOFMAGICANDSUBIUGATEALLWIZARDSANDMUGGLES
```

Task 2.5. Polyalphabetic Ciphers – Vigenère



Bước 1: Chuẩn hóa dữ liệu đầu vào (normalize)

Mục đích: Để đảm bảo phép tính mã hóa hoạt động chính xác, dữ liệu đầu vào cần được chuẩn hóa về cùng một kiểu chữ. Trong chương trình, toàn bộ ký tự được chuyển về chữ thường (lowercase) trước khi xử lý.

Logic code

- Duyệt từng ký tự trong chuỗi văn bản.
- Sử dụng hàm tolower() để chuyển ký tự sang chữ thường.

- Trả về chuỗi đã chuẩn hóa.

```
Vigenere.cpp > main()
#include <iostream>
#include <string>
#include <cctype>

using namespace std;

// —— Chuẩn hóa dữ liệu: chuyển về chữ thường ——
string normalize(string text) {
    for (char &c : text)
        c = tolower(c);
    return text;
}
```

Bước 2: Sinh chuỗi khóa lặp lại

Mục đích: Trong thuật toán Vigenère, khóa phải có độ dài tương ứng với số ký tự alphabet trong bản rõ. Vì vậy cần tạo một chuỗi khóa được lặp lại nhiều lần cho đến khi đạt độ dài cần thiết.

Logic code

- Chuẩn hóa khóa về chữ thường.
- Duyệt từng ký tự trong plaintext.
- Nếu ký tự là chữ cái:
    - Lấy ký tự khóa tương ứng bằng phép toán modulo.
    - Thêm vào chuỗi khóa mới.
- Nếu không phải chữ cái thì giữ nguyên vị trí.

```
// ── Sinh chuỗi khóa lặp lại ──
string generateKey(string text, string key) {
    key = normalize(key);
    string newKey = "";
    int keyIndex = 0;

    for (char c : text) {
        if (isalpha(c)) {
            newKey += key[keyIndex % key.length()];
            keyIndex++;
        } else {
            newKey += c;
        }
    }
    return newKey;
}
```

Bước 3: Xây dựng hàm mã hóa (Encrypt)

Theo công thức của Vigenère cipher:

Ci = (Pi + Ki) \mod26

Trong đó:

- Pi : ký tự bản rõ
- Ki : ký tự khóa
- Ci : ký tự bản mã

Logic code

- Chuẩn hóa plaintext.
- Sinh chuỗi khóa lặp lại bằng hàm generateKey.
- Duyệt từng ký tự:
    - Nếu là chữ cái:
        - Chuyển ký tự sang giá trị số (0–25).
        - Áp dụng công thức (pi + ki) % 26.
        - Chuyển lại thành ký tự chữ cái.
    - Nếu không phải chữ cái thì giữ nguyên.

Ví dụ code:

```
// —— Hàm mã hóa ——
// Ci = (pi + ki) mod 26
string encrypt(string plaintext, string key) {
    plaintext = normalize(plaintext);
    string fullKey = generateKey(plaintext, key);

    string cipher = "";

    for (int i = 0; i < plaintext.length(); i++) {
        char p = plaintext[i];

        if (isalpha(p)) {
            int pi = p - 'a';
            int ki = fullKey[i] - 'a';

            int ci = (pi + ki) % 26;

            cipher += char('A' + ci);
        } else {
            cipher += p;
        }
    }

    return cipher;
}
```

Bước 4: Xây dựng hàm giải mã (Decrypt)

Theo công thức:

Pi=(Ci−Ki+26)mod26

Logic code

- Chuẩn hóa ciphertext.
- Sinh chuỗi khóa lặp lại.
- Duyệt từng ký tự:
    - Nếu là chữ cái:
        - Chuyển về giá trị số.
        - Áp dụng công thức giải mã.
        - Chuyển lại thành ký tự chữ cái.
    - Nếu không phải chữ cái thì giữ nguyên.

```
// ── Hàm giải mã ──
// pi = (Ci - ki + 26) mod 26
string decrypt(string cipher, string key) {
    cipher = normalize(cipher);
    string fullKey = generateKey(cipher, key);

    string plain = "";

    for (int i = 0; i < cipher.length(); i++) {
        char c = cipher[i];

        if (isalpha(c)) {
            int ci = c - 'a';
            int ki = fullKey[i] - 'a';

            int pi = (ci - ki + 26) % 26;

            plain += char('a' + pi);
        } else {
            plain += c;
        }
    }

    return plain;
}
```

Bước 5: Chương trình chính (Main)

Chức năng

- Cho phép người dùng lựa chọn:
  - 1 → mã hóa văn bản
  - 2 → giải mã văn bản
- Nhập plaintext/ciphertext và khóa.
- Gọi hàm encrypt() hoặc decrypt().
- In kết quả ra màn hình.

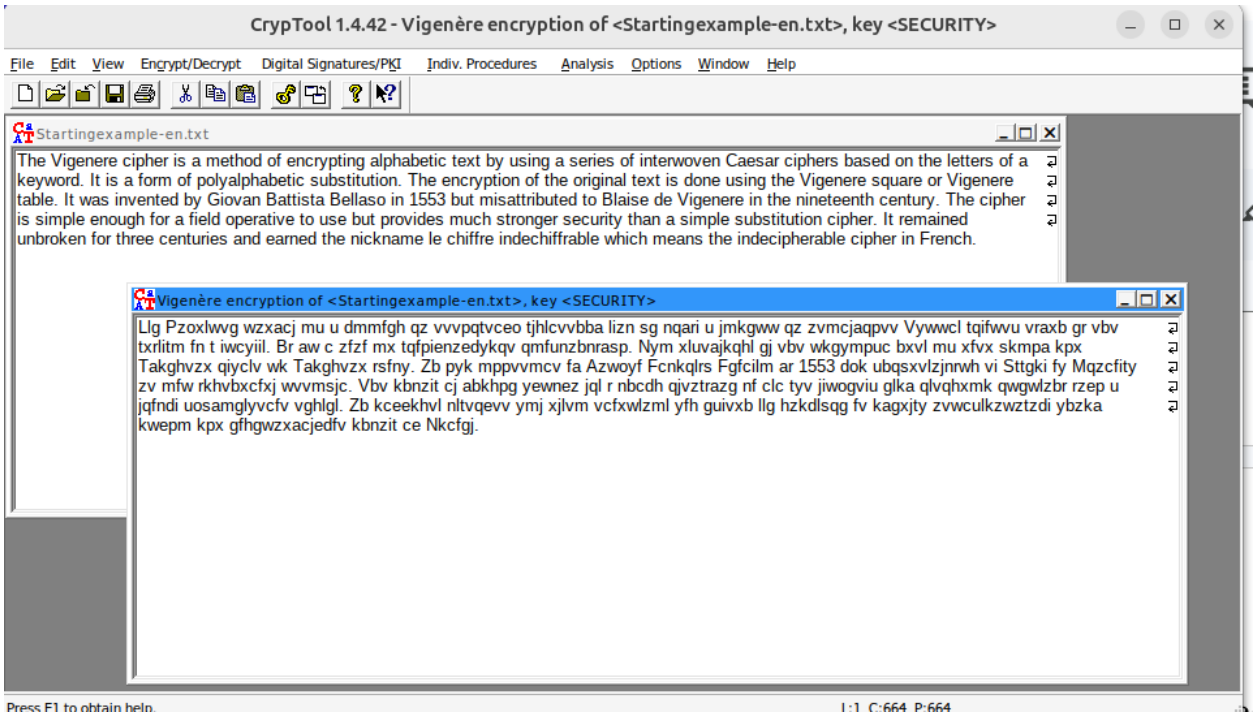
```
int main() {

    int choice;
    string text, key;

    cout << "===== Vigenere Cipher =====\n";
    cout << "1. Encrypt\n";
    cout << "2. Decrypt\n";
    cout << "Choose: ";
    cin >> choice;
    cin.ignore();
    if (choice == 1) {

        cout << "Enter plaintext:\n";
        getline(cin, text);

        cout << "Enter key: ";
        getline(cin, key);

        string cipher = encrypt(text, key);

        cout << "\nCiphertext:\n" << cipher << endl;
    }
    else if (choice == 2) {

        cout << "Enter ciphertext:\n";
        getline(cin, text);

        cout << "Enter key: ";
        getline(cin, key);
        string plain = decrypt(text, key);
        cout << "\nDecrypted text:\n" << plain << endl;
    }
    return 0;
}
```

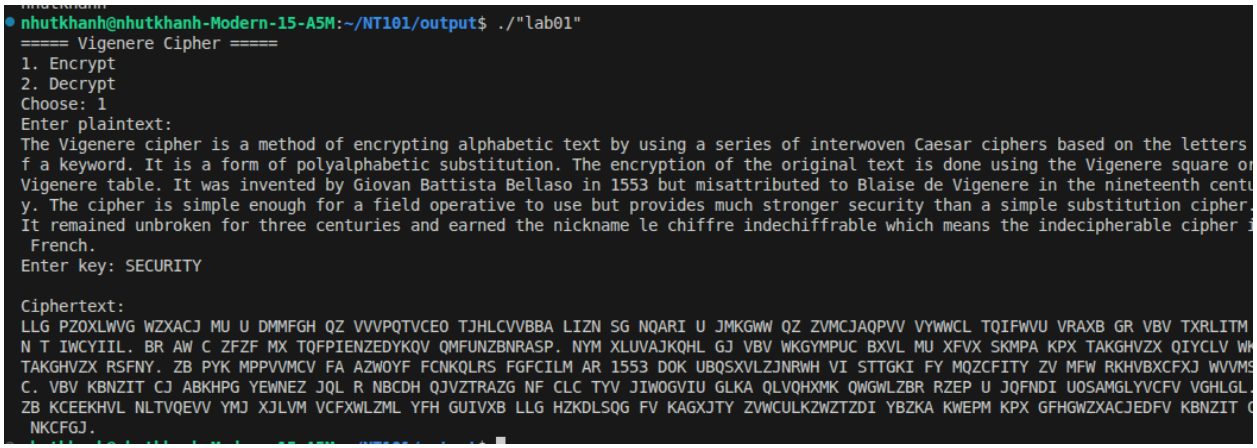
- Kết quả kiểm thử

Chương trình được kiểm thử với một văn bản dài hơn 100 từ. Sau khi mã hóa và giải mã lại bằng cùng khóa, kết quả thu được trùng khớp với bản rõ ban đầu, chứng tỏ thuật toán được cài đặt chính xác.

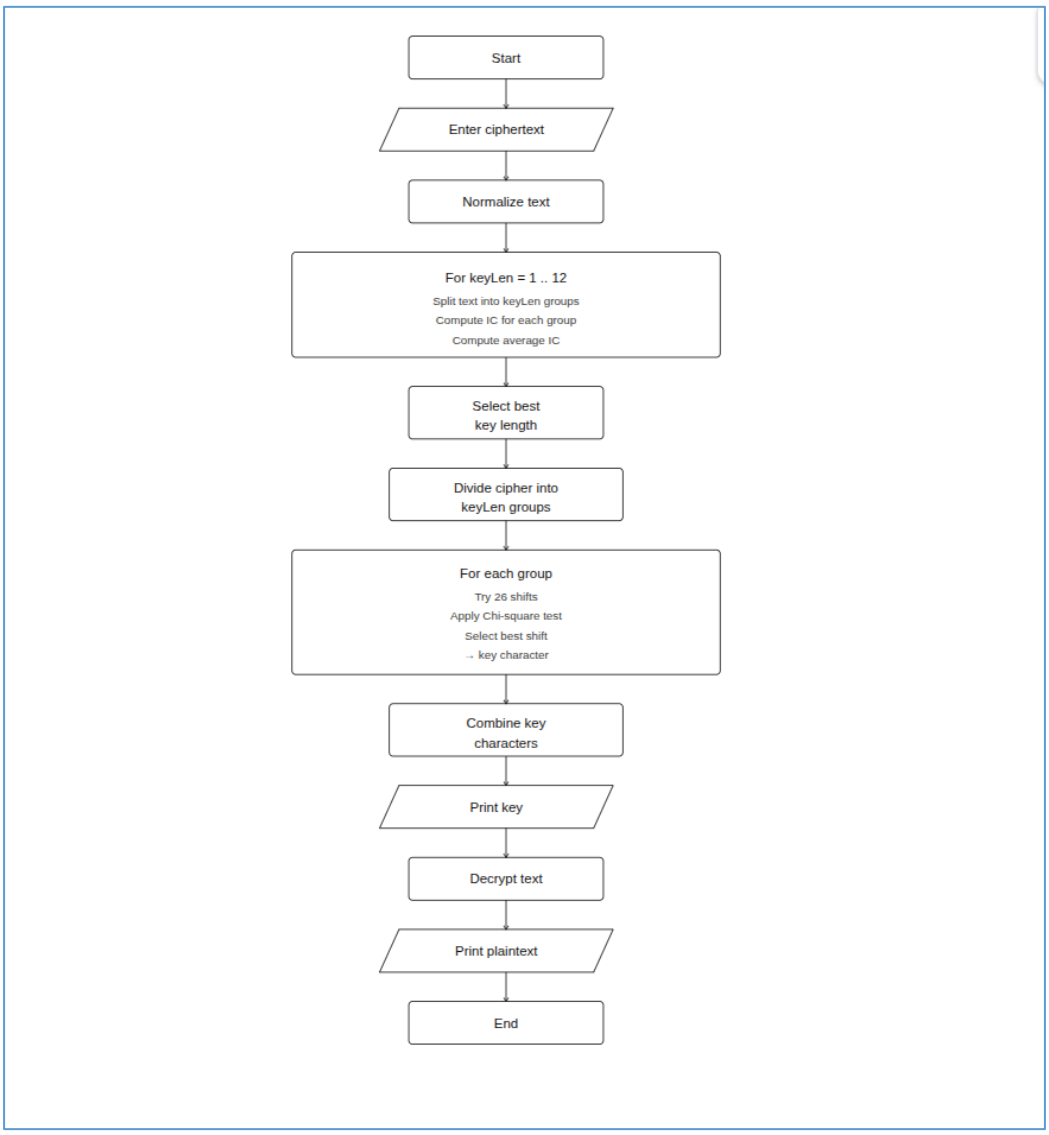
Ngoài ra, kết quả ciphertext cũng được so sánh với các công cụ online như:

- Cryptii
- CrypTool


CrypTool 1.4.42 - Vigenère encryption of <Startingexample-en.txt>, key <SECURITY>
File Edit View Encrypt/Decrypt Digital Signatures/PKI Indiv. Procedures Analysis Options Window Help
Startingexample-en.txt
The Vigenere cipher is a method of encrypting alphabetic text by using a series of interwoven Caesar ciphers based on the letters of a keyword. It is a form of polyalphabetic substitution. The encryption of the original text is done using the Vigenere square or Vigenere table. It was invented by Giovan Battista Bellaso in 1553 but misattributed to Blaise de Vigenere in the nineteenth century. The cipher is simple enough for a field operative to use but provides much stronger security than a simple substitution cipher. It remained unbroken for three centuries and earned the nickname le chiffre indechiffrable which means the indecipherable cipher in French.
Vigenère encryption of <Startingexample-en.txt>, key <SECURITY>
Llg Pzoxlwvg wzxacj mu u dmmfgh qz vvvpqtvceo tjhlcvvbba lizn sg nqari u jmkgww qz zvmcjaqpvv Vywwcl tqifwvu vraxb gr vbv txrlitm fn t iwcyiil. Br aw c zfzf mx tqfpienzedykqv qmfunzbnrasp. Nym xluvajkqhl gj vbv wkgympuc bxvl mu xfvx skmpa kpx Takghvzx qiyclv wk Takghvzx rsfny. Zb pyk mppvvmcv fa Azwoyf Fcnkqlrs Fgfcilm ar 1553 dok ubqsxvlzjnrwh vi Sttgki fy Mqzcfity zv mfw rkhvbxcfxj wvvmsjc. Vbv kbnzit cj abkhpg yewnez jql r nbcdh qjvztrazg nf clc tyv jiwogviu glka qlvqhxmk qwgwlzbr rzep u jqfndi uosamglyvcfv vghlgl. Zb kceekhvl nltvqevv ymj xjlvm vcfxwlzml yfh guivxb llg hzkdlsqg fv kagxjty zvwculkzwztzdi ybzka kwepm kpx gfhgwzxacjedfv kbnzit ce Nkcfgj.
Press F1 to obtain help.
L:1 C:664 P:664


nhutkhanh@nhutkhanh-Modern-15-A5M:~/NT101/output$ ./"lab01"
===== Vigenere Cipher =====
1. Encrypt
2. Decrypt
Choose: 1
Enter plaintext:
The Vigenere cipher is a method of encrypting alphabetic text by using a series of interwoven Caesar ciphers based on the letters
f a keyword. It is a form of polyalphabetic substitution. The encryption of the original text is done using the Vigenere square or
Vigenere table. It was invented by Giovan Battista Bellaso in 1553 but misattributed to Blaise de Vigenere in the nineteenth centu
y. The cipher is simple enough for a field operative to use but provides much stronger security than a simple substitution cipher.
It remained unbroken for three centuries and earned the nickname le chiffre indechiffrable which means the indecipherable cipher i
French.
Enter key: SECURITY
Ciphertext:
LLG PZOXLWVG WZXACJ MU U DMMFGH QZ VVVPQTVCEO TJHLCVVBBA LIZN SG NQARI U JMKGWW QZ ZVMCJAQPVV VYWWCL TQIFWVU VRAXB GR VBV TXRLITM
N T IWCYIIL. BR AW C ZFZF MX TQFPIENZEDYKQV QMFUNZBNRASP. NYM XLUVAJKQHL GJ VBV WKGYMPUC BXVL MU XFVX SKMPA KPX TAKGHVZX QIYCLV WK
TAKGHVZX RSFNY. ZB PYK MPPVVMCV FA AZWOYF FCNKQLRS FGFCILM AR 1553 DOK UBQSXVLZJNRWH VI STTGKI FY MQZCFITY ZV MFW RKHVBXCFXJ WVVMS
C. VBV KBNZIT CJ ABKHPG YEWNEZ JQL R NBCDH QJVZTRAZG NF CLC TYV JIWOGVIU GLKA QLVQHXMK QWGWLZBR RZEP U JQFNDI UOSAMGLYVCFV VGHLGL
ZB KCEEKHVL NLTVQEVV YMJ XJLVM VCFXWLZML YFH GUIVXB LLG HZKDLSQG FV KAGXJTY ZVWCULKZWZTZDI YBZKA KWEPM KPX GFHGWZXACJEDFV KBNZIT C
NKCFGJ.

Task 2.6. Breaking the Vigenère Cipher



1. Mục tiêu

Trong nhiệm vụ này, chương trình được xây dựng nhằm phá mã Vigenère cipher khi không biết trước khóa. Chương trình cho phép người dùng nhập một ciphertext bất kỳ, sau đó thực hiện các bước phân tích thống kê để ước lượng độ dài khóa, tìm khóa phù hợp và giải mã văn bản.

Do Vigenère cipher sử dụng nhiều phép dịch chuyển khác nhau nên việc phân tích tần suất đơn giản không còn hiệu quả. Tuy nhiên, bằng cách sử dụng Index of Coincidence (IC) và chi-square analysis, ta vẫn có thể suy ra khóa và khôi phục bản rõ.

2. Các bước thực hiện

2.1 Chuẩn hóa dữ liệu đầu vào

Trước khi tiến hành phân tích, ciphertext cần được chuẩn hóa để đảm bảo việc tính toán thống kê chính xác. Chương trình thực hiện các thao tác:

- Chuyển toàn bộ ký tự về chữ thường
- Loại bỏ các ký tự không phải alphabet

Điều này giúp dữ liệu đầu vào chỉ còn các ký tự từ a đến z.

Đoạn code thực hiện chuẩn hóa:

```
string normalize(string text) {
    string result="";
    for(char c:text)
        if(isalpha(c))
            result+=tolower(c);
    return result;
}
```

2.2 Tính Index of Coincidence

Index of Coincidence (IC) được sử dụng để đo mức độ trùng lặp của các ký tự trong văn bản. Giá trị IC của văn bản tiếng Anh thường khoảng 0.065, trong khi văn bản ngẫu nhiên khoảng 0.038.

Công thức tính IC:

$$IC = \frac{\sum f_i(f_i - 1)}{N(N-1)}$$

Trong đó:

- Fi là số lần xuất hiện của ký tự thứ i
- N là tổng số ký tự trong văn bản

Đoạn code tính IC:

```
double indexOfCoincidence(string text){

    int freq[26]={0};

    for(char c:text)
        freq[c-'a']++;

    int N=text.size();

    double num=0;

    for(int i=0;i<26;i++)
        num+=freq[i]*(freq[i]-1);

    return num/(N*(N-1.0));
}

```

2.3 Ước lượng độ dài khóa

Để tìm độ dài khóa, chương trình thử các giá trị từ 1 đến 12. Với mỗi độ dài khóa giả định, ciphertext được chia thành nhiều nhóm ký tự. Sau đó tính IC trung bình của các nhóm. Độ dài khóa được chọn là giá trị có IC trung bình gần nhất với 0.065.

Đoạn code thực hiện:

```
int findKeyLength(string text){

    int bestLen=1;
    double bestDiff=1e9;
    double target=0.065;

    for(int len=1;len<=12;len++){

        double avgIC=0;

        for(int i=0;i<len;i++){

            string group="";

            for(int j=i;j<text.size();j+=len)    comparison of integer expr
                group+=text[j];

            avgIC+=indexOfCoincidence(group);
        }

        avgIC/=len;

        double diff=abs(avgIC-target);

        if(diff<bestDiff){
            bestDiff=diff;
            bestLen=len;
        }
    }

    return bestLen;
}
```

2.4 Phân tích tần suất để tìm từng ký tự của khóa

Sau khi biết độ dài khóa mmm, ciphertext được chia thành mmm nhóm ký tự. Mỗi nhóm tương đương với một Caesar cipher.

Để tìm ký tự khóa cho từng nhóm, chương trình sử dụng chi-square test nhằm so sánh tần suất ký tự của nhóm với tần suất tiếng Anh.

Đoạn code tính chi-square:

```
double chiSquare(string text){

    int freq[26]={0};

    for(char c:text)
        freq[c-'a']++;

    int n=text.size();

    double chi=0;

    for(int i=0;i<26;i++){

        double expected=n*englishFreq[i]/100;

        chi+=pow(freq[i]-expected,2)/expected;
    }

    return chi;
}
```

Sau đó chương trình thử 26 phép dịch chuyển khác nhau và chọn giá trị có chi-square nhỏ nhất.

Code tìm ký tự khóa:

```
char findKeyChar(string group){

    double bestScore=1e18;
    int bestShift=0;

    for(int s=0;s<26;s++){

        string decoded="";

        for(char c:group)
            decoded+=char('a'+(c-'a'-s+26)%26);

        double sc=chiSquare(decoded);

        if(sc<bestScore){
            bestScore=sc;
            bestShift=s;
        }
    }

    return 'a'+bestShift;
```

2.5 Xây dựng khóa hoàn chỉnh

Sau khi tìm được ký tự khóa cho từng nhóm, các ký tự này được ghép lại để tạo thành khóa hoàn chỉnh.

Đoạn code:

```
}

string findKey(string text,int keyLen){

    string key="";

    for(int i=0;i<keyLen;i++){

        string group="";

        for(int j=i;j<text.size();j+=keyLen)    comparison of int
            group+=text[j];

        key+=findKeyChar(group);
    }

    return key;
}
```

2.6 Giải mã ciphertext

Sau khi tìm được khóa, ciphertext được giải mã theo công thức Vigenère:

$$p_i = (c_i - k_i + 26) \mod 26$$

Đoạn code thực hiện giải mã:

```
string decrypt(string text,string key){

    string plain="";
    int m=key.size();

    for(int i=0;i<text.size();i++){    comparison of integer ex

        int ci=text[i]-'a';
        int ki=key[i%m]-'a';

        plain+=char('a'+(ci-ki+26)%26);
    }

    return plain;
}
```

3. Kết quả thực nghiệm

Sau khi chạy chương trình với ciphertext được cung cấp "pp oiuibvql avpgzwm, vyabnwzycbbg klhqla mv uqwckl kzzwktcfcwg hpp wwftwzcktakah bxjjzcynlu pyzbcgp zzht omnpxtcfckts eahkxwve uvw h uqn wy ywxy-jtzgp wiejwxubbvpe wiesgp utzvtunpfz va nztuurizf tgemizlu uh etfu fbim htq bikk va xmvprtyz. mogey lxagdgqgpufck tsialqmooe uzx buqx nhy edsxmviduxape wyg zlpqlimpqz uvw kkscbts uuavbui mhl oltuzqvhvuiv mv rdibxjv pubt wtupivf, yqv jkvyecvz vp fbm buvqlvxa czx khuhuxmgakmf khtoghqvhvuivl zwob il jtqxqm jcdx bkhpeukmpqzm igk gyuqe" , kết quả thu được:

Cipher length: 408

Estimated key length: 6

Recovered key: hcmuit

Sau khi giải mã: in computer science cryptography refers to secure information and communication techniques derived from mathematical concepts and algorithms...

```
nhutkhanh@nhutkhanh-Modern-15-A5M:~/NT101/output$ ./"task2.6_lab01"
Enter ciphertext (end with empty line):
pp oiuibvql avpgzwm, vyabnwzycbbg klhqla mv uqwckl kzzwktcfcwg hppwwftwzcktakah bxjjzcynlu pyzbcgp zzht omnpxtcfckts eahkxwve uvwh
uqn wy ywxy-jtzgp wiejwxubbvpe wiesgp utzvtunpfz va nztuurizftgemizlu uh etfu fbim htq bikk va xmvprtyz. mogey lxagdgqgpufcktsialqm
ooe uzx buqx nhy edsxmviduxape wyg zlpqlimpqz uvw kkscbtsuuavbui mhl oltuzqvhvuiv mv rdibxjv pubt wtupivf, yqv jkvyecvz vpfbm buvql
vxa czx khuhuxmgakmf khtoghqvhvuivl zwob il jtqxqm jcdxbkhpeukmpqzm igk gyuqe


Cipher length: 408
Estimated key length: 6
Recovered key: hcmuit

Decrypted plaintext:
incomputersciencecryptographyreferstosecureinformationandcommunicationtechniquesderivedfrommathematicalconceptsandasetofrulebasedca
lculationscalledalgorithmstotransformmessagesinwaysthatarehardtodecipherthesedeterministicalgorithmsareusedforcryptographickeygener
ationanddigitalsigningandverificationtoprotectdataprivacywebbrowsingontheinternetandconfidentialcommunicationssuchascreditcardtrans
actionsandemail
```

Plaintext thu được có ý nghĩa rõ ràng trong tiếng Anh, chứng tỏ chương trình phá mã thành công.

4. Kết luận

Trong bài thực hành này, chương trình đã được xây dựng để tự động phá mã Vigenère cipher khi không biết khóa ban đầu. Bằng cách sử dụng Index of Coincidence để ước lượng độ dài khóa và chi-square analysis để tìm từng ký tự khóa, chương trình có thể khôi phục lại plaintext với độ chính xác cao.

Kết quả cho thấy các phương pháp thống kê có thể phá vỡ các hệ mã cổ điển như Vigenère cipher khi ciphertext đủ dài.

Task 2.7 – Rail Fence cipher

1. Giới thiệu thuật toán

Rail Fence cipher là một dạng transposition cipher (mật mã hoán vị). Thuật toán không thay đổi ký tự, chỉ thay đổi vị trí của các ký tự trong văn bản.

Ý tưởng:

- Viết plaintext theo **dạng zigzag** trên nhiều dòng (rails)
- Sau đó đọc lần lượt **từng dòng từ trên xuống**

Ví dụ với **3 rails**

Plaintext: HELLOWORLD

Viết zigzag:

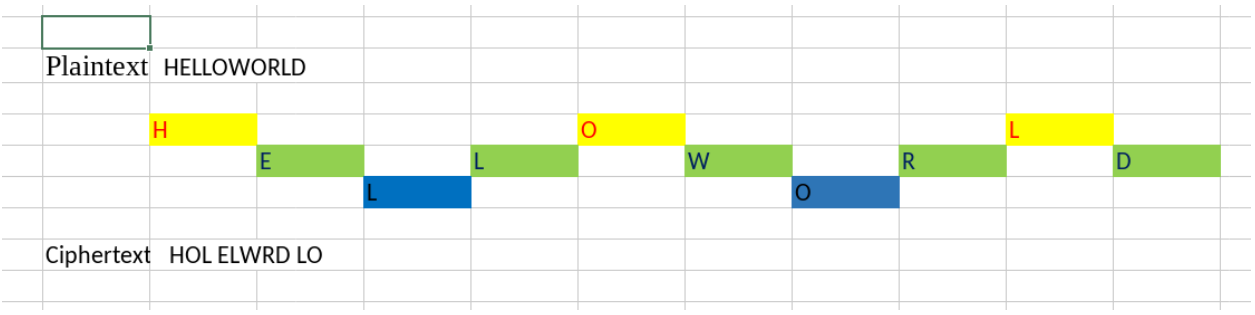
H O L

E L W R D

L O

Đọc từng dòng: HOL ELWRD LO

Ciphertext: HOLELWRDLO



2. Nguyên lý hoạt động

2.1 Mã hóa

Các bước:

1. Chọn **số hàng (key)**
2. Viết plaintext theo **zigzag**
3. Đọc từng hàng để tạo ciphertext

2.2 Giải mã

Các bước:

1. Tạo ma trận zigzag rỗng
2. Đánh dấu vị trí ký tự
3. Điền ciphertext vào
4. Đọc zigzag để lấy plaintext

3. Cài đặt chương trình

Chương trình được cài đặt bằng ngôn ngữ C++ và bao gồm ba thành phần chính: hàm mã hóa, hàm giải mã và chương trình chính để điều khiển quá trình nhập xuất dữ liệu.

3.1 Hàm chuẩn hóa dữ liệu normalize

Hàm normalizeText() được xây dựng nhằm chuẩn hóa dữ liệu đầu vào trước khi thực hiện mã hóa.

```cpp
Rail_Fence_cipher.cpp > ...
#include <iostream>
#include <vector>
#include <string>
#include <cctype>

using namespace std;

/*----------------------------------------
  Ham chuan hoa du lieu
  Chuyen tat ca ky tu ve chu thuong
----------------------------------------*/
string normalize(string text) {
    string result = "";
    for(char &c : text) {
        if(!isspace(c)) {
            result += toupper(c);
        }
    }
    return result;
}
```

3.2 Hàm mã hóa encryptRailFence()

Hàm encryptRailFence() có nhiệm vụ thực hiện quá trình **mã hóa văn bản theo thuật toán Rail Fence**. Hàm nhận vào hai tham số là chuỗi văn bản cần mã hóa (plaintext) và khóa key đại diện cho số hàng được sử dụng trong mô hình zigzag.

Trong quá trình hoạt động, hàm sẽ phân bố các ký tự của plaintext vào các hàng khác nhau theo dạng zigzag. Khi quá trình phân bố hoàn tất, các ký tự trong từng hàng sẽ được đọc lần lượt từ trên xuống dưới để tạo thành chuỗi ciphertext. Chuỗi kết quả này chính là bản mã sau khi áp dụng thuật toán Rail Fence.

```cpp
/*----------------------------------------
  Ham ma hoa Rail Fence
----------------------------------------*/
string encryptRailFence(string text, int key) {

    vector<string> rail(key);

    int row = 0;
    bool down = true;

    for(char c : text) {

        rail[row] += c;

        if(row == 0)
            down = true;
        else if(row == key - 1)
            down = false;

        row += down ? 1 : -1;
    }

    string result;

    for(int i = 0; i < key; i++)
        result += rail[i];

    return result;
}
```

3.2 Hàm giải mã decryptRailFence()

Hàm decryptRailFence() thực hiện quá trình **khôi phục bản rõ từ bản mã** dựa trên cùng một khóa đã sử dụng trong quá trình mã hóa.

Hàm trước tiên tái tạo lại cấu trúc zigzag tương ứng với độ dài của ciphertext và số hàng của khóa. Sau đó chương trình xác định vị trí các ký tự trong ma trận zigzag và điền các ký tự của ciphertext vào đúng vị trí đó. Cuối cùng, chương trình đọc lại các ký tự theo đường zigzag ban đầu để thu được chuỗi plaintext. Chuỗi này chính là nội dung văn bản ban đầu trước khi mã hóa.

```
/*---------------------------------------
  Ham giai ma Rail Fence
---------------------------------------*/
string decryptRailFence(string cipher, int key) {

    int n = cipher.length();
    vector<vector<char>> rail(key, vector<char>(n, '\n'));

    bool down;
    int row = 0, col = 0;

    // Danh dau zigzag
    for(int i = 0; i < n; i++) {

        if(row == 0) down = true;
        if(row == key - 1) down = false;

        rail[row][col++] = '*';
        row += down ? 1 : -1;
    }

    // Dien ciphertext vao bang
    int index = 0;

    for(int i = 0; i < key; i++)
        for(int j = 0; j < n; j++)
            if(rail[i][j] == '*' && index < n)
                rail[i][j] = cipher[index++];

    // Doc zigzag de lay plaintext
    string result;

    row = 0;
    col = 0;

    for(int i = 0; i < n; i++) {

        if(row == 0) down = true;
        if(row == key - 1) down = false;

        result += rail[row][col++];
        row += down ? 1 : -1;
    }

    return result;
}

```

3.3 Chương trình chính main()

Hàm main() đóng vai trò **điều khiển toàn bộ chương trình và tương tác với người dùng**. Khi chương trình chạy, người dùng có thể lựa chọn giữa hai chức năng là mã hóa hoặc giải mã.

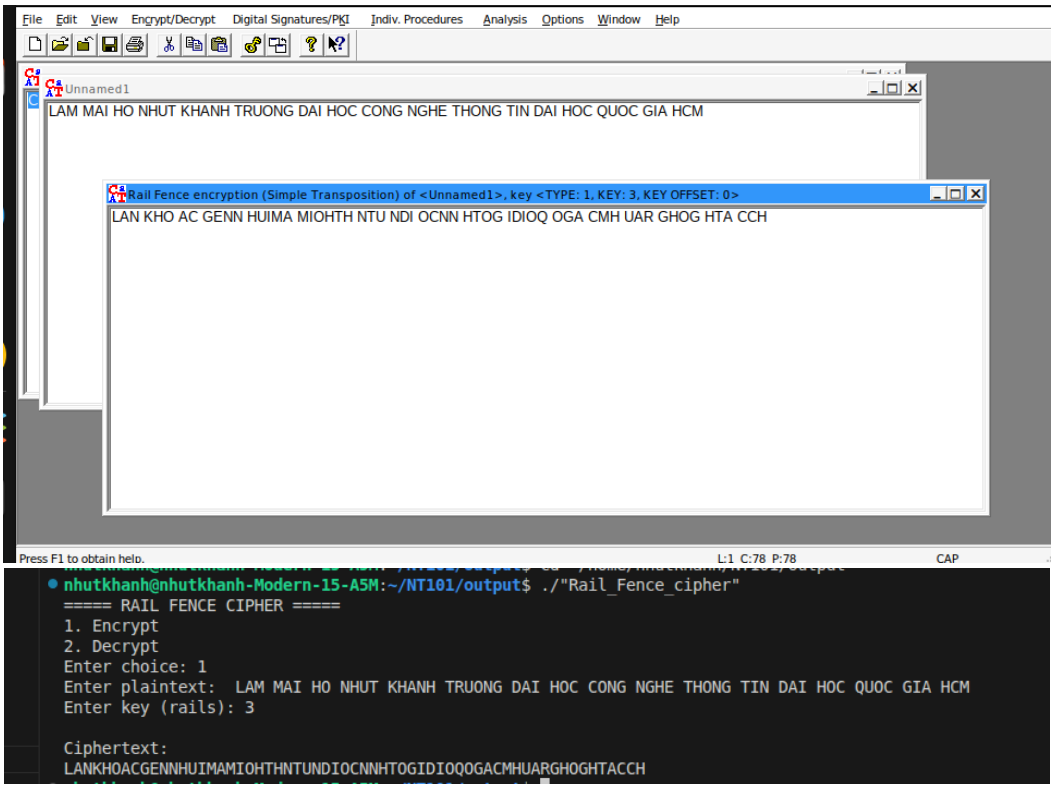
Nếu người dùng chọn chức năng mã hóa, chương trình sẽ yêu cầu nhập plaintext và khóa, sau đó gọi hàm encryptRailFence() để thực hiện mã hóa và hiển thị kết quả ciphertext. Ngược lại, nếu người dùng chọn chức năng giải mã, chương trình sẽ yêu cầu nhập ciphertext và khóa, sau đó gọi hàm decryptRailFence() để khôi phục lại plaintext.

Nhờ cấu trúc này, chương trình có thể thực hiện đầy đủ hai chức năng chính của thuật toán Rail Fence là **mã hóa và giải mã văn bản**.

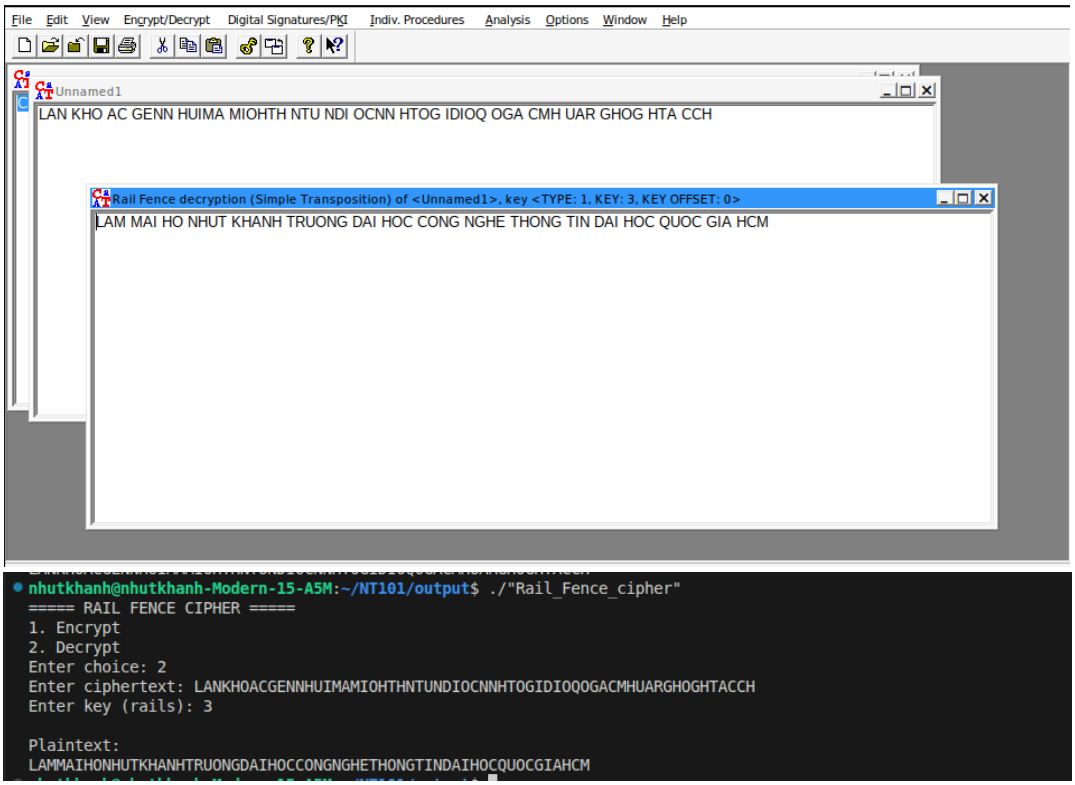
```
int main() {

    int choice, key;
    string text;

    cout << "===== RAIL FENCE CIPHER =====\n";
    cout << "1. Encrypt\n";
    cout << "2. Decrypt\n";
    cout << "Enter choice: ";
    cin >> choice;

    cin.ignore();

    if(choice == 1) {

        cout << "Enter plaintext: ";
        getline(cin, text);

        text = normalize(text);

        cout << "Enter key (rails): ";
        cin >> key;

        string cipher = encryptRailFence(text, key);

        cout << "\nCiphertext:\n" << cipher << endl;
    }

    else if(choice == 2) {

        cout << "Enter ciphertext: ";
        getline(cin, text);

        cout << "Enter key (rails): ";
        cin >> key;

        string plain = decryptRailFence(text, key);

        cout << "\nPlaintext:\n" << plain << endl;
    }

    else {

        cout << "Invalid choice!" << endl;
    }

    return 0;
}
```

Kết quả chạy chương trình:

Mã hóa:



Giải mã:



HẾT